
Linear Algebra for Machine Learning

Refresher Module 2 — Vectors, Transforms, Eigen & SVD

vector spaces & bases · dot products & projection · Gram–Schmidt · linear transformations · the transformation zoo · change of basis · eigen/SVD · PCA · PageRank

Computing, Robotics & Bionics

3rd-year Bioengineering — companion to A. Géron, *Hands-On Machine Learning with Scikit-Learn and PyTorch*

*This is the second of four refresher modules. It builds the linear algebra that machine learning runs on from the ground up: vector spaces and their axioms, linear combinations, span and basis, and change of basis for both vectors and maps — worked out not just for $\mathbb{R}^n$ but for polynomials and matrices too, to show the machinery is the same wherever it is applied; the geometric layer — the dot product, projection, and Gram–Schmidt orthonormalization; linear transformations defined abstractly and then represented by matrices (scaling, rotation, reflection, shear, projection), orthonormal matrices and rigid motions; inversion, the determinant, rank, and solving $\mathbf{Ax} = \mathbf{b}$; and the two factorisations that power ML — the eigendecomposition and the singular value decomposition — ending in least squares and the pseudoinverse that every regression fit actually calls, regularisation read through the eigenvalues, principal-component analysis, and PageRank as an eigenvector-centrality case study. It is **self-contained**: together with its three companion Colab notebooks — a bioengineering-adapted superset of Géron’s `math_linear_algebra` material — it is your complete reference, so you never read two versions side by side. The same rotations and orthonormal frames appear in the companion Robotics & Bionics unit ($SO(3)$, homogeneous transforms); projection underlies least squares (Géron Ch. 4) and the eigen/SVD machinery underlies PCA (Ch. 7). Each section ends with exercises.*

Prerequisites: Module 1 (NumPy); high-school algebra and trigonometry. Software: a web browser and a Google account (everything runs in Google Colab; nothing to install).

Contents

1	Vectors and Vector Spaces	2
2	Linear Combinations, Span, and Basis	2
3	Row and Column Vectors: Coordinates in the Canonical Basis	2
4	Change of Basis for Vectors	3
5	Other Vector Spaces: Polynomials and Matrices	5
5.1	Polynomials of Degree at Most Two	5
5.2	2×2 Matrices	6
6	The Scalar Product, Length, and Orthonormalization	7
6.1	The scalar product in a general basis	7
6.2	Orthonormal bases and the dot product formula	8
6.3	Norms: what counts as a length	8
6.4	Geometric meaning: angle, orthogonality, cosine similarity	9
6.5	Projection	10
6.6	Gram–Schmidt orthonormalization	10
7	Linear Transformations and Matrices	11
7.1	What makes a map <i>linear</i>	11
7.2	Columns are images of the basis vectors	12
7.3	Composition is matrix multiplication	12
8	Change of Basis for Linear Transformations	12
8.1	Same Matrix, Different Meaning: Basis Change vs. Linear Map	13
8.2	Choosing an Easy Basis: the Conjugation Trick	14
9	The Transformation Zoo	17
10	Orthogonal Matrices and Rigid Motions	18
11	Inversion, Determinant, Rank, and Solving $Ax = b$	18
11.1	Solving a linear system	18
12	Eigenvalues and Eigenvectors	19
13	The Singular Value Decomposition	19
14	Two Payoffs: Least Squares and PCA	20
14.1	Linear regression is a projection	20
14.2	PCA is an eigen/SVD problem	21
15	PageRank: An Eigenvector-Centrality Case Study	23
15.1	Importance as a fixed point	23
15.2	Finding r : power iteration	23
15.3	Dead ends, traps and damping	23
16	The Lab, and Where This Leads in Géron	24

1 Vectors and Vector Spaces

Machine learning represents a sample as a **vector** of features and a model as a set of **matrices**; training and prediction are then linear-algebra operations. This module builds the geometric picture behind them.

Before fixing any coordinates, it is worth being precise about what a vector actually is. A **vector space** V over $\mathbb{R}$ is a set equipped with two operations, vector addition and scalar multiplication, satisfying ten axioms:

Definition

For all $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V$ and scalars $c, d \in \mathbb{R}$: *Addition* — closure ($\mathbf{u} + \mathbf{v} \in V$), commutativity ($\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$), associativity ($(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$), an additive identity $\mathbf{0}$ with $\mathbf{v} + \mathbf{0} = \mathbf{v}$, and an additive inverse $-\mathbf{v}$ with $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$. *Scalar multiplication* — closure ($c\mathbf{v} \in V$), distributivity over vector addition ($c(\mathbf{u} + \mathbf{v}) = c\mathbf{u} + c\mathbf{v}$) and over scalar addition ($(c + d)\mathbf{v} = c\mathbf{v} + d\mathbf{v}$), compatibility ($c(d\mathbf{v}) = (cd)\mathbf{v}$), and the scalar identity ($1\mathbf{v} = \mathbf{v}$).

Nothing here mentions arrays of numbers: 2×2 matrices under matrix addition and scalar multiplication satisfy every one of these axioms (their zero vector is the all-zero matrix), and so do polynomials of bounded degree or continuous functions under pointwise addition. Every idea in this module — basis, linear map, eigenvector — applies to spaces like these verbatim. We will specialise almost immediately to the one vector space this module actually computes with, $\mathbb{R}^n$ with the usual component-wise addition and scaling, but the abstraction earns its keep later: a weight matrix, a gradient, even a linear transformation itself, all turn out to be “just a vector” in some vector space once you need them to be.

2 Linear Combinations, Span, and Basis

A **linear combination** of vectors $\mathbf{v}_1, \dots, \mathbf{v}_k$ is any vector $c_1\mathbf{v}_1 + \dots + c_k\mathbf{v}_k$ formed from them with scalar weights. The **span** of a set of vectors is the set of *all* their linear combinations — everything reachable by scaling and adding them.

Definition

A set of vectors $\mathbf{v}_1, \dots, \mathbf{v}_k$ is **linearly independent** if the only linear combination equal to the zero vector is the trivial one,

$$c_1\mathbf{v}_1 + \dots + c_k\mathbf{v}_k = \mathbf{0} \implies c_1 = \dots = c_k = 0.$$

Equivalently, no vector in the set is a linear combination of the others — none is redundant. A **basis** of a vector space V is a linearly independent set that spans V .

Every basis of V has the same number of vectors, called the **dimension** of V . A basis is the minimal amount of information needed to reach every vector in V by linear combination, and — crucially — it is not unique: $\mathbb{R}^2$ has infinitely many bases, from the standard $(1, 0)^\top, (0, 1)^\top$ to any pair of non-parallel vectors. Which basis you choose does not change the space; it changes how conveniently you can compute in it, a theme the rest of this module returns to repeatedly.

3 Row and Column Vectors: Coordinates in the Canonical Basis

Once a basis $\mathbf{e}_1, \dots, \mathbf{e}_n$ of V is fixed, every vector $\mathbf{v} \in V$ has a unique expression as a linear combination of it, $\mathbf{v} = \sum_i v_i \mathbf{e}_i$, and the list of weights $(v_1, \dots, v_n)$ — its **coordinates** — is all the information needed to recover $\mathbf{v}$. This is the precise sense in which “a vector is a list of numbers”: the list is never absolute, it is always relative to a chosen basis.

In $\mathbb{R}^n$ the **canonical (standard) basis** is $\mathbf{e}_1 = (1, 0, \dots, 0)^\top, \dots, \mathbf{e}_n = (0, \dots, 0, 1)^\top$, and writing a vector as the column array $\mathbf{x} = (x_1, \dots, x_n)^\top$ is exactly writing its coordinates in this basis — so ubiquitous that we normally suppress it and just say “the vector $\mathbf{x}$ ”, but every array-of-numbers computation you do is secretly a statement about coordinates in $\mathbf{e}_1, \dots, \mathbf{e}_n$.

Row times column is bookkeeping, not the point

A *row* vector $\mathbf{x}^\top$ and a *column* vector $\mathbf{x}$ hold identical coordinates; the distinction is a convention for writing matrix products, not a property of the vector itself. In NumPy this is visible directly: a 1-D array has no row or column orientation at all, and `x.T` does nothing to it. The row/column bookkeeping only becomes necessary once you commit to representing a vector as a literal $n \times 1$ or $1 \times n$ matrix — which is exactly what happens once we compute the scalar product between two vectors as an ordinary matrix product (Section 6).

Shapes, and how the book stores your data

Two conventions will be in force at once for the rest of the course, and knowing which is which prevents most of the errors you will meet in the lab.

First, the mechanical rule: an $m \times n$ matrix times an $n \times p$ matrix gives an $m \times p$ one, and the product only exists when the *inner* dimensions match. Reading a formula by its shapes is the fastest way to tell whether it can possibly be right.

Second, the layout. In this module — and in Section 7, where a matrix becomes a map — a vector is a *column* and a matrix acts on it from the left, $\mathbf{y} = \mathbf{A}\mathbf{x}$. But Scikit-Learn, Géron and every dataset you will load store the data matrix $\mathbf{X}$ with **one sample per row** and one feature per column, so $\mathbf{X}$ is $m \times n$ for m samples and n features. A linear model’s predictions for the whole dataset are then $\mathbf{X}\mathbf{w}$, an $m \times 1$ column of one number per sample, and a single prediction is the dot product of one *row* of $\mathbf{X}$ with $\mathbf{w}$. The transposes that appear when you compare a textbook formula with working code are almost always this, and nothing deeper. It is also why $\mathbf{X}^\top \mathbf{X}$ — which you will meet in Section 14.1 and Section 14.2 — is $n \times n$, one entry per pair of features, rather than one per pair of samples.

4 Change of Basis for Vectors

A vector’s coordinates depend entirely on which basis you measure them in — the same abstract vector looks like a different list of numbers in a different basis. Let $B = \{\mathbf{b}_1, \dots, \mathbf{b}_n\}$ and $E = \{\mathbf{e}_1, \dots, \mathbf{e}_n\}$ be two bases of the *same* vector space V — not necessarily the canonical one: Section 5 runs this same machinery on polynomials and matrices, where there is no “the” canonical basis to fall back on. Write each vector of E as a linear combination of the vectors of B and collect the coefficients, one E -vector per column, into the **basis-change matrix**

$$\mathbf{\Gamma}_{BE} = \begin{bmatrix} [\mathbf{e}_1]_B & [\mathbf{e}_2]_B & \cdots & [\mathbf{e}_n]_B \end{bmatrix}, \quad (4.1)$$

where $[\mathbf{e}_j]_B$ denotes $\mathbf{e}_j$ ’s coordinate column in basis B . This is the notation used throughout the companion notebook, and the naming is mnemonic: reading the subscript left to right, $\mathbf{\Gamma}_{BE}$ expresses the vectors of the second (last) letter as coordinates in the basis of the first letter.

Key idea

If $\mathbf{v}_B$ denotes a vector’s coordinates in basis B and $\mathbf{v}_E$ its coordinates in basis E , then

$$\mathbf{v}_B = \mathbf{\Gamma}_{BE} \mathbf{v}_E. \quad (4.2)$$

This is immediate from the definition of coordinates (Section 3): $\mathbf{v} = \sum_j (v_E)_j \mathbf{e}_j = \sum_j (v_E)_j \sum_i (\mathbf{\Gamma}_{BE})_{ij} \mathbf{b}_i$, which written as a single matrix equation is exactly (4.2). The matrix runs “the other way” on purpose — it *takes in* E -coordinates and *returns* B -coordinates — because its columns are built *from* the E -vectors expressed *in* B . The *same* vector, two coordinate lists; nothing about the vector itself has changed.

A basis-change matrix is always square and invertible (its columns are a basis, hence linearly independent), so translating in the opposite direction uses the inverse:

$$\mathbf{\Gamma}_{EB} = (\mathbf{\Gamma}_{BE})^{-1}, \quad \mathbf{v}_E = \mathbf{\Gamma}_{EB} \mathbf{v}_B. \quad (4.3)$$

Worked example

Take E to be the canonical basis of $\mathbb{R}^2$ and $B = \{\mathbf{b}_1, \mathbf{b}_2\}$ with $\mathbf{b}_1 = (1, 1)^\top$, $\mathbf{b}_2 = (1, -1)^\top$ (both written, as they must be to build $\mathbf{\Gamma}_{EB}$, in E -coordinates). Stacking them as columns,

$$\mathbf{\Gamma}_{EB} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{\Gamma}_{BE} = (\mathbf{\Gamma}_{EB})^{-1} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

The vector with E -coordinates $\mathbf{v}_E = (3, 1)^\top$ has B -coordinates

$$\mathbf{v}_B = \mathbf{\Gamma}_{BE} \mathbf{v}_E = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix},$$

and you can check the round trip: $\mathbf{\Gamma}_{EB} \mathbf{v}_B = 2\mathbf{b}_1 + 1\mathbf{b}_2 = (3, 1)^\top = \mathbf{v}_E$. Figure 1 draws this same vector against both grids — one square, one skewed — to make the point visually: it is one arrow, measured two ways.

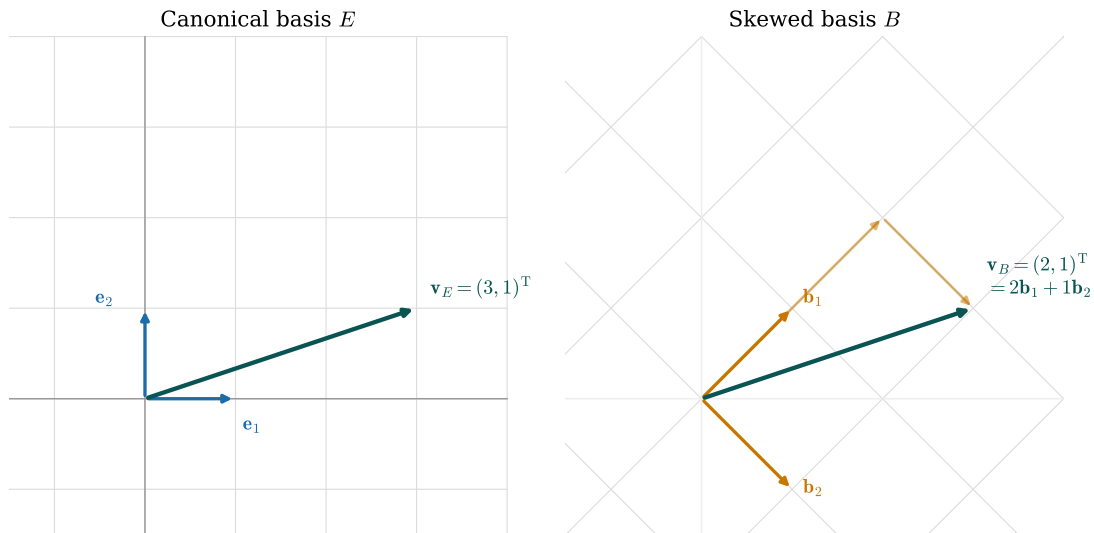


Figure 1: The same vector $\mathbf{v}$ with $\mathbf{v}_E = (3, 1)^\top$, plotted against the canonical grid E (left, where it is just the arrow to the point $(3, 1)$) and against the skewed grid of the worked example’s basis B (right, where the very same arrow decomposes as $2\mathbf{b}_1 + 1\mathbf{b}_2$, i.e. $\mathbf{v}_B = (2, 1)^\top$). Same arrow, two coordinate stories.

Exercises

1. Is the set of vectors in $\mathbb{R}^2$ with both coordinates positive a vector space under the usual operations? Which axiom fails first?
2. Show that 2×2 matrices under matrix addition and scalar multiplication satisfy the vector-space axioms; what is the zero vector of this space?
3. Are $(1, 2, 2)^\top, (2, 1, -2)^\top, (2, -2, 1)^\top$ linearly independent? Do they form a basis of $\mathbb{R}^3$?
4. Let E be the canonical basis of $\mathbb{R}^3$ and $B = \{\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3\}$ with $\mathbf{b}_1 = \mathbf{e}_1$, $\mathbf{b}_2 = \mathbf{e}_1 + \mathbf{e}_2$, $\mathbf{b}_3 = \mathbf{e}_1 + \mathbf{e}_2 + \mathbf{e}_3$. Write $\mathbf{\Gamma}_{EB}$, then find $\mathbf{v}_E$ for the vector with $\mathbf{v}_B = (1, -1, 1)^\top$.
5. Compute $\mathbf{\Gamma}_{BE}$ by inverting $\mathbf{\Gamma}_{EB}$ from the previous item, and use it to recover $\mathbf{v}_B$ from your answer above.

5 Other Vector Spaces: Polynomials and Matrices

Section 1 promised that matrices and polynomials satisfy the vector-space axioms just as well as $\mathbb{R}^n$ does. It is time to cash that promise in: two concrete, non- $\mathbb{R}^n$ vector spaces, each given two bases, with $\mathbf{\Gamma}_{BE}$ built and used exactly as in Section 4. The point of doing this is not the arithmetic (it is identical arithmetic) — it is seeing that “basis” and “coordinates” are properties of *any* vector space, and that the machinery you just learned for arrows in $\mathbb{R}^n$ travels unchanged to functions and to matrices.

5.1 Polynomials of Degree at Most Two

Let $V = P_2$, the polynomials $p(x) = a_0 + a_1x + a_2x^2$ of degree at most two — a vector space of dimension 3 under the usual addition and scalar multiplication of polynomials. Take

$$E = \{1, x, x^2\}, \quad B = \{1, 1+x, 1+x+x^2\}.$$

E plays the role of the canonical basis: a polynomial’s E -coordinates are just its coefficients, $[a_0 + a_1x + a_2x^2]_E = (a_0, a_1, a_2)^\top$. Expressing each vector of B in E -coordinates and stacking as columns,

$$\mathbf{\Gamma}_{EB} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{\Gamma}_{BE} = (\mathbf{\Gamma}_{EB})^{-1} = \begin{bmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix}. \quad (5.1)$$

Key idea

This is *exactly* the matrix from the companion notebook’s own abstract three-vector example, where one basis is built from another the same cumulative way — $1, 1+x, 1+x+x^2$ here; three abstract vectors there. Nothing about $\mathbf{\Gamma}_{EB}$ knows or cares whether the basis vectors are arrows, polynomials, or anything else — it is pure bookkeeping of coordinates.

Worked example

Take $p(x) = 1 + x^2$, so $p_E = (1, 0, 1)^\top$ directly from its coefficients. Its B -coordinates are

$$p_B = \mathbf{\Gamma}_{BE} p_E = \begin{bmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix},$$

i.e. $p(x) = 1 \cdot 1 - 1 \cdot (1+x) + 1 \cdot (1+x+x^2)$ — check it: the constants are $1 - 1 + 1 = 1$, the x terms are $0 - 1 + 1 = 0$, the x^2 term is $0 - 0 + 1 = 1$, recovering $1 + 0x + x^2$. Figure 2

draws this out term by term.

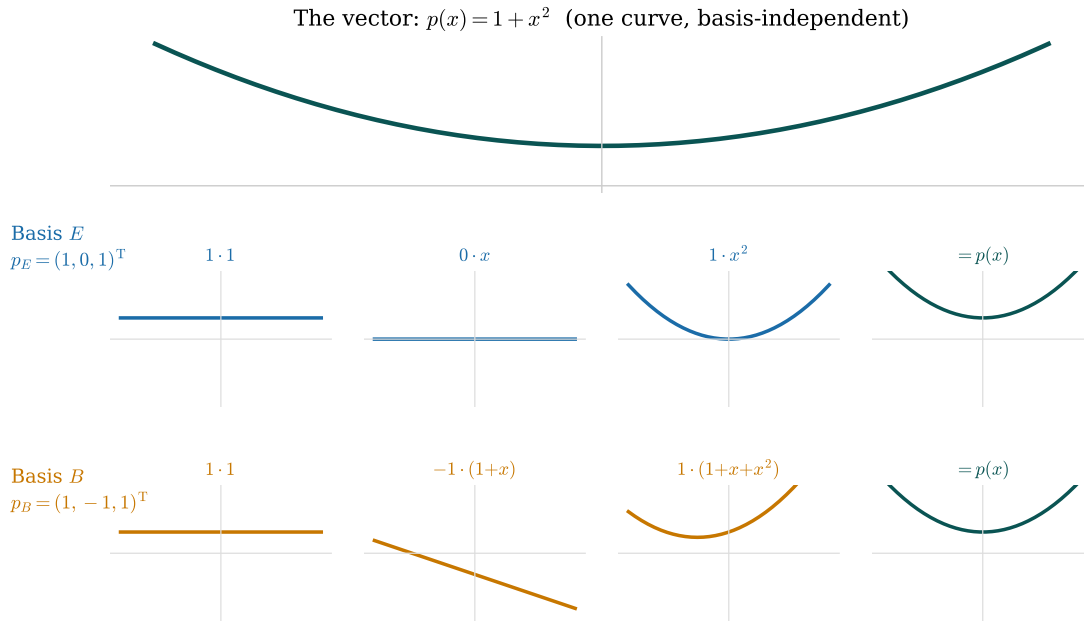


Figure 2: The polynomial $p(x) = 1 + x^2$ is one curve (top) — basis-independent. Underneath, two different decompositions into scaled basis functions both sum to that same curve: $p_E = (1, 0, 1)^T$ in $E = \{1, x, x^2\}$ (middle row) and $p_B = (1, -1, 1)^T$ in $B = \{1, 1+x, 1+x+x^2\}$ (bottom row).

Exercises

1. Confirm $\mathbf{\Gamma}_{BE}$ in (5.1) really is the inverse of $\mathbf{\Gamma}_{EB}$ by multiplying them together.
2. Take $q(x) = 2 - x + 3x^2$. Write q_E directly, then compute q_B . Check your answer by expanding q_B 's combination of $1, 1+x, 1+x+x^2$ back out.
3. Is $\{1, 1-x, 1-x^2\}$ a basis of P_2 ? If so, find its $\mathbf{\Gamma}_{EB}$ too.

5.2 2×2 Matrices

Let $V = \mathbb{R}^{2 \times 2}$, the 2×2 real matrices — dimension 4, as you already found in Section 4's exercises. The canonical-style basis is the four “unit matrices,”

$$E = \{\mathbf{E}_{11}, \mathbf{E}_{12}, \mathbf{E}_{21}, \mathbf{E}_{22}\}, \quad \mathbf{E}_{11} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \mathbf{E}_{12} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \mathbf{E}_{21} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}, \mathbf{E}_{22} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix},$$

with a matrix's E -coordinates being its four entries read off in that order, $\begin{bmatrix} a & b \\ c & d \end{bmatrix}_E = (a, b, c, d)^T$. A second, more meaningful basis splits every matrix into its **symmetric** and **antisymmetric** parts (recall $\mathbf{A} = \mathbf{A}^T$ symmetric matrices from the spectral theorem, Section 12):

$$B = \{\mathbf{S}_1, \mathbf{S}_2, \mathbf{S}_3, \mathbf{S}_4\}, \quad \mathbf{S}_1 = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \mathbf{S}_2 = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}, \mathbf{S}_3 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \mathbf{S}_4 = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}.$$

$\mathbf{S}_1, \mathbf{S}_2, \mathbf{S}_3$ are symmetric, $\mathbf{S}_4$ is antisymmetric, and every 2×2 matrix is uniquely a combination of the two ($\mathbf{M} = \frac{1}{2}(\mathbf{M} + \mathbf{M}^T) + \frac{1}{2}(\mathbf{M} - \mathbf{M}^T)$ — the same symmetric/antisymmetric split behind the Gram matrix of Section 6.1). Reading each $\mathbf{S}_i$'s entries off in the same (a, b, c, d) order and

stacking as columns,

$$\mathbf{\Gamma}_{EB} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{\Gamma}_{BE} = (\mathbf{\Gamma}_{EB})^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & \frac{1}{2} & -\frac{1}{2} & 0 \end{bmatrix}. \quad (5.2)$$

Worked example

Take $\mathbf{M} = \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix}$, so $M_E = (2, 3, 1, 4)^\top$ directly. Then

$$M_B = \mathbf{\Gamma}_{BE} M_E = (2, 4, 2, 1)^\top,$$

i.e. $\mathbf{M} = 2\mathbf{S}_1 + 4\mathbf{S}_2 + 2\mathbf{S}_3 + 1\mathbf{S}_4$. You can read this straight off the symmetric/antisymmetric split: $\mathbf{M}_{\text{sym}} = \begin{bmatrix} 2 & 2 \\ 2 & 4 \end{bmatrix} = 2\mathbf{S}_1 + 4\mathbf{S}_2 + 2\mathbf{S}_3$ and $\mathbf{M}_{\text{antisym}} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} = 1 \cdot \mathbf{S}_4$. Figure 3 shows both decompositions as heatmaps.

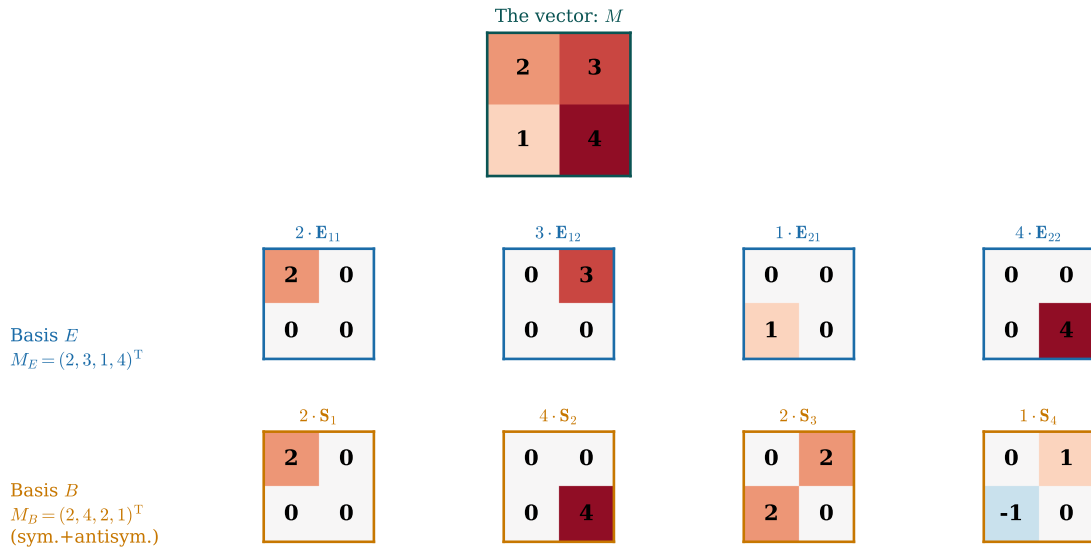


Figure 3: The matrix $\mathbf{M}$ is one 2×2 array of numbers — basis-independent. Its unit-matrix decomposition $M_E = (2, 3, 1, 4)^\top$ (middle row) and its symmetric/antisymmetric decomposition $M_B = (2, 4, 2, 1)^\top$ (bottom row) both sum, cell by cell, back to $\mathbf{M}$.

Exercises

1. Verify $\mathbf{\Gamma}_{EB}\mathbf{\Gamma}_{BE} = \mathbf{I}$ for (5.2) by multiplying the two matrices.
2. Take $\mathbf{N} = \begin{bmatrix} 0 & 5 \\ -1 & 2 \end{bmatrix}$. Find N_E and N_B , and check that the symmetric part you get from N_B matches $\frac{1}{2}(\mathbf{N} + \mathbf{N}^\top)$ computed directly.
3. Which entry of M_B or N_B would change if the matrix had been antisymmetric to begin with? Why does that make sense given what $\mathbf{S}_4$ represents?

6 The Scalar Product, Length, and Orthonormalization

6.1 The scalar product in a general basis

The **scalar (inner) product** $\langle \mathbf{v}, \mathbf{w} \rangle$ is a number measuring how two vectors relate — their alignment and, as we will see, their lengths — and it must be computable from coordinates

alone. Expanding both vectors bilinearly in a basis $\mathbf{e}_1, \dots, \mathbf{e}_n$ (Section 3),

$$\langle \mathbf{v}, \mathbf{w} \rangle = \left\langle \sum_i v_i \mathbf{e}_i, \sum_j w_j \mathbf{e}_j \right\rangle = \sum_{i=1}^n \sum_{j=1}^n v_i w_j \langle \mathbf{e}_i, \mathbf{e}_j \rangle = \mathbf{v}^T \mathbf{G} \mathbf{w}, \quad (6.1)$$

Definition

The matrix $\mathbf{G}$ is the basis's **Gram matrix**, $G_{ij} = \langle \mathbf{e}_i, \mathbf{e}_j \rangle$ — it records how every basis vector relates to every other one, and it is exactly the extra information the raw coordinates v_i, w_j do not carry.

If the basis is *not* orthonormal, $\mathbf{G}$ is some matrix other than the identity, and the scalar product is *not* $\sum_i v_i w_i$ — forgetting this is one of the most common sources of confusion in linear algebra. Equation (6.1) is the general rule; every formula in the rest of this module is the special case that follows next.

6.2 Orthonormal bases and the dot product formula

A basis is **orthonormal** when its vectors are mutually perpendicular (*ortho*) and each has unit length (*normal*) — the two halves of the word, made precise in Section 6.3 and Section 6.4 respectively, though the one-line condition below needs neither: $\langle \mathbf{e}_i, \mathbf{e}_j \rangle = \delta_{ij}$ (1 if $i = j$, 0 otherwise), i.e. exactly when $\mathbf{G} = \mathbf{I}$. Equation (6.1) then collapses to

$$\langle \mathbf{v}, \mathbf{w} \rangle = \mathbf{v}^T \mathbf{w} = \sum_{i=1}^n v_i w_i, \quad (6.2)$$

the familiar **dot product** — a consequence of orthonormality, not a definition in its own right. $\mathbb{R}^n$'s standard basis $(1, 0, \dots, 0), \dots, (0, \dots, 0, 1)$ is orthonormal, so whenever we write coordinates in it (as we do throughout this module unless said otherwise), (6.2) is exactly the right formula, and we will call $\mathbf{v} \cdot \mathbf{w} = \mathbf{v}^T \mathbf{w}$ “the dot product” from here on.

6.3 Norms: what counts as a length

“Length” deserves a definition of its own, because a vector space admits more than one and machine learning uses several. A **norm** is any rule $\|\cdot\|$ assigning a number to each vector and obeying three requirements.

Definition

A **norm** on a vector space satisfies, for all vectors $\mathbf{v}, \mathbf{w}$ and every scalar c :

1. **Positivity.** $\|\mathbf{v}\| \geq 0$, and $\|\mathbf{v}\| = 0$ only for $\mathbf{v} = \mathbf{0}$ — nothing but the zero vector has zero length.
2. **Homogeneity.** $\|c\mathbf{v}\| = |c| \|\mathbf{v}\|$ — scaling a vector scales its length by the same factor.
3. **Triangle inequality.** $\|\mathbf{v} + \mathbf{w}\| \leq \|\mathbf{v}\| + \|\mathbf{w}\|$ — going direct is never longer than going via a detour.

Any rule satisfying all three is a legitimate notion of length, and the **distance** between two points is then $\|\mathbf{v} - \mathbf{w}\|$.

The scalar product supplies one for free. Taking a vector's product with itself and square-rooting,

$$\|\mathbf{v}\| = \sqrt{\langle \mathbf{v}, \mathbf{v} \rangle}, \quad (6.3)$$

gives the norm *induced* by the inner product, and in an orthonormal basis (6.2) turns it into the **Euclidean** or **L2 norm**

$$\|\mathbf{x}\|_2 = \sqrt{x_1^2 + \dots + x_n^2} = \sqrt{\mathbf{x}^\top \mathbf{x}}. \quad (6.4)$$

Two others earn their keep in ML, both read off the coordinates directly: the **L1 norm** $\|\mathbf{x}\|_1 = \sum_i |x_i|$, which Lasso uses to regularise, and the **L ∞ norm** $\|\mathbf{x}\|_\infty = \max_i |x_i|$. All three belong to the family $\|\mathbf{x}\|_p = (\sum_i |x_i|^p)^{1/p}$ at $p = 1, 2$ and the limit $p \rightarrow \infty$. A vector of length one is a **unit vector**, $\hat{\mathbf{x}} = \mathbf{x}/\|\mathbf{x}\|_2$, and dividing by the norm like this is called **normalising**.

Key idea

Only the L2 norm comes from an inner product, and that single fact decides what geometry is available. A norm is induced by some scalar product exactly when it obeys the *parallelogram law* $\|\mathbf{v} + \mathbf{w}\|^2 + \|\mathbf{v} - \mathbf{w}\|^2 = 2\|\mathbf{v}\|^2 + 2\|\mathbf{w}\|^2$. Test it on $\mathbf{v} = (1, 0)^\top$ and $\mathbf{w} = (0, 1)^\top$: L2 gives $4 = 4$, while L1 gives $8 \neq 4$ and L ∞ gives $2 \neq 4$. So L1 and L ∞ measure length perfectly well but carry no notion of angle — everything from Section 6.4 onwards (angles, orthogonality, projection, least squares, Gram–Schmidt) is a privilege of the inner product, not of merely having a length.

The three norms disagree about which vectors count as short, and the shape of disagreement has consequences. Drawing each **unit ball** $\{\mathbf{x} : \|\mathbf{x}\| \leq 1\}$:

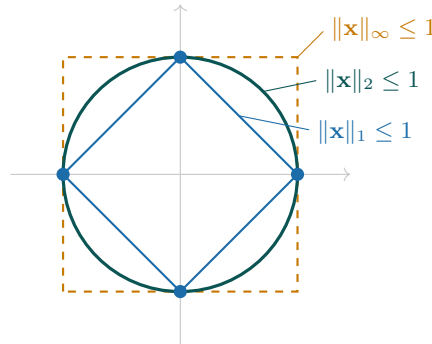


Figure 4: The unit ball of each norm. All three agree on the axes and disagree everywhere else: along the diagonal the L1 boundary sits at $(\frac{1}{2}, \frac{1}{2})$, the L2 boundary at $(0.707, 0.707)$ and the L ∞ boundary at $(1, 1)$. Only the round one comes from a scalar product. The marked corners of the L1 diamond lie *on* the axes, which is the geometric reason an L1 penalty (Lasso, Section 14.1) drives coefficients exactly to zero while the smooth L2 ball (Ridge) only shrinks them.

6.4 Geometric meaning: angle, orthogonality, cosine similarity

The dot product’s geometric meaning is the key to everything that follows:

$$\mathbf{x}^\top \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta, \quad (6.5)$$

where θ is the angle between the vectors. Three consequences matter:

- **Orthogonality.** $\mathbf{x}^\top \mathbf{y} = 0$ exactly when the vectors are perpendicular ($\cos 90^\circ = 0$). Orthogonality is just “dot product zero”.
- **Angle / similarity.** Rearranging (6.5) gives the **cosine similarity** $\cos \theta = \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$, the standard measure of how aligned two feature vectors (or two gene-expression profiles, or two documents) are.

- **Sign.** The dot product is positive when vectors point the same way, negative when opposed — the basis of every linear classifier’s decision $\mathbf{w}^\top \mathbf{x} + b \gtrless 0$.

Derived, or defined?

In $\mathbb{R}^2$ or $\mathbb{R}^3$, (6.5) is a *theorem*, not a definition: with θ the ordinary, protractor-measured angle, the Law of Cosines gives $\|\mathbf{x} - \mathbf{y}\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{y}\|^2 - 2\|\mathbf{x}\|\|\mathbf{y}\|\cos\theta$, and expanding $\|\mathbf{x} - \mathbf{y}\|^2 = \mathbf{x}^\top \mathbf{x} - 2\mathbf{x}^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}$ algebraically and comparing the two gives (6.5) straight back out. The genuinely surprising content is that the coordinate sum $\sum_i x_i y_i$ — which never mentions an angle — equals the classical geometric quantity $\|\mathbf{x}\|\|\mathbf{y}\|\cos\theta$, and this only works because the coordinates are orthonormal (Section 6.2). Once there is no protractor to appeal to — $\mathbb{R}^n$ for $n \geq 4$, or any abstract inner-product space — there is no independent angle left to derive a formula *for*. There, (6.5) is turned around and used as the **definition** of θ ,

$$\cos\theta := \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\| \|\mathbf{y}\|},$$

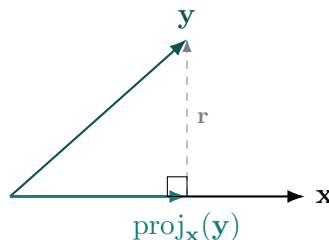
which only makes sense because the **Cauchy–Schwarz inequality** $|\langle \mathbf{x}, \mathbf{y} \rangle| \leq \|\mathbf{x}\|\|\mathbf{y}\|$ guarantees that ratio always lies in $[-1, 1]$, so arccos of it is always well defined.

6.5 Projection

Decomposing one vector along another is the geometric heart of least squares. The **orthogonal projection** of $\mathbf{y}$ onto the line through $\mathbf{x}$ is

$$\text{proj}_{\mathbf{x}}(\mathbf{y}) = \frac{\mathbf{x}^\top \mathbf{y}}{\mathbf{x}^\top \mathbf{x}} \mathbf{x} = (\hat{\mathbf{x}}^\top \mathbf{y}) \hat{\mathbf{x}}, \quad (6.6)$$

and the leftover $\mathbf{r} = \mathbf{y} - \text{proj}_{\mathbf{x}}(\mathbf{y})$ is **orthogonal** to $\mathbf{x}$. So any vector splits uniquely into a part along $\mathbf{x}$ and a part perpendicular to it.



Key idea

Projection onto a *subspace* (a plane, or the column space of a matrix) works the same way and is exactly how linear regression finds the best fit: it projects the target vector onto the space spanned by the feature columns, making the residual orthogonal to every feature. We derive that “normal equation” in Section 14.1, where (14.2) turns the projection into a single matrix and (6.6) reappears as its one-column special case.

6.6 Gram–Schmidt orthonormalization

An orthonormal basis — mutually perpendicular unit vectors — is the most convenient basis there is: as Section 6.2 showed, coordinates are dot products *exactly because* $\mathbf{G} = \mathbf{I}$ for such a basis, and (Section 10) the change-of-basis matrix built from it is orthogonal, so its inverse is free. Most bases you are handed are not orthonormal, but **Gram–Schmidt** turns any linearly

independent set $\mathbf{v}_1, \dots, \mathbf{v}_k$ into an orthonormal set $\mathbf{q}_1, \dots, \mathbf{q}_k$ spanning the same subspace, using nothing but projection, one vector at a time:

$$\mathbf{q}_1 = \frac{\mathbf{v}_1}{\|\mathbf{v}_1\|}, \quad \mathbf{u}_k = \mathbf{v}_k - \sum_{j=1}^{k-1} (\mathbf{q}_j^\top \mathbf{v}_k) \mathbf{q}_j, \quad \mathbf{q}_k = \frac{\mathbf{u}_k}{\|\mathbf{u}_k\|}. \quad (6.7)$$

Each $\mathbf{u}_k$ is $\mathbf{v}_k$ with its projection (6.6) onto every previously built axis $\mathbf{q}_1, \dots, \mathbf{q}_{k-1}$ subtracted off — exactly the “along/perpendicular” split of Section 6.5, applied repeatedly against a growing subspace instead of a single line. What is left, $\mathbf{u}_k$, is perpendicular to everything built so far; normalising it gives the next axis.

Key idea

Stacking the $\mathbf{q}_k$ as columns of a matrix $\mathbf{Q}$ and recording the projection coefficients from (6.7) in an upper-triangular $\mathbf{R}$ gives the **QR decomposition** $\mathbf{A} = \mathbf{Q}\mathbf{R}$ of the matrix whose columns were $\mathbf{v}_1, \dots, \mathbf{v}_k$. Solving the least-squares normal equation of Section 14 via QR ($\mathbf{R}\mathbf{w} = \mathbf{Q}^\top \mathbf{y}$) avoids ever forming $\mathbf{X}^\top \mathbf{X}$, and is the numerically stable way real solvers do it.

Exercises

1. Let $\mathbf{e}_1 = (1, 0)^\top$, $\mathbf{e}_2 = (1, 1)^\top$ be a (non-orthonormal) basis of $\mathbb{R}^2$. Write out its Gram matrix $\mathbf{G}$, then compute $\langle \mathbf{v}, \mathbf{w} \rangle$ for $\mathbf{v} = 2\mathbf{e}_1 + \mathbf{e}_2$, $\mathbf{w} = \mathbf{e}_1 - \mathbf{e}_2$ two ways: via $\mathbf{v}^\top \mathbf{G} \mathbf{w}$ in $\mathbf{e}_1, \mathbf{e}_2$ coordinates, and via the ordinary dot product after writing $\mathbf{v}, \mathbf{w}$ out in standard coordinates. Confirm the two answers agree.
2. For $\mathbf{x} = (3, 4)^\top$, compute $\|\mathbf{x}\|_2$, $\|\mathbf{x}\|_1$, $\|\mathbf{x}\|_\infty$ and the unit vector $\hat{\mathbf{x}}$.
3. Are $(1, 2, 2)^\top$ and $(2, 1, -2)^\top$ orthogonal? Compute their cosine similarity.
4. Project $\mathbf{y} = (2, 3)^\top$ onto $\mathbf{x} = (1, 0)^\top$ and onto $\mathbf{x} = (1, 1)^\top$; give the residual in each case and verify it is orthogonal.
5. Explain in one sentence why a linear classifier’s rule $\mathbf{w}^\top \mathbf{x} + b > 0$ is a statement about the angle between $\mathbf{w}$ and $\mathbf{x}$.
6. Apply Gram–Schmidt (6.7) to $\mathbf{v}_1 = (1, 1, 0)^\top$, $\mathbf{v}_2 = (1, 0, 1)^\top$ to find an orthonormal basis $\mathbf{q}_1, \mathbf{q}_2$ for their span, and confirm $\mathbf{q}_1^\top \mathbf{q}_2 = 0$.
7. Explain why solving the Section 14 normal equation via a QR factorisation is preferred in practice to inverting $\mathbf{X}^\top \mathbf{X}$ directly.

7 Linear Transformations and Matrices

7.1 What makes a map *linear*

A function $T : V \rightarrow W$ between vector spaces is a **linear transformation** if it respects both vector-space operations:

Definition

T is linear when, for all $\mathbf{u}, \mathbf{v} \in V$ and scalars α, β ,

$$T(\alpha \mathbf{u} + \beta \mathbf{v}) = \alpha T(\mathbf{u}) + \beta T(\mathbf{v}). \quad (7.1)$$

Geometrically this means T preserves lines through the origin, scaling, and vector addition — it cannot bend space, only stretch, rotate, reflect, shear or flatten it. The remarkable fact that

makes matrices useful at all: *every* linear transformation between finite-dimensional spaces is, once bases are chosen for V and W , exactly multiplication by some matrix $\mathbf{A}$ — so from here on “linear transformation” and “matrix” are two names for the same thing.

7.2 Columns are images of the basis vectors

The single most useful fact: *the columns of $\mathbf{A}$ are where it sends the standard basis vectors*. In 2-D, $\mathbf{A}\mathbf{e}_1$ is the first column and $\mathbf{A}\mathbf{e}_2$ the second, because

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} | & | \\ \mathbf{a}_1 & \mathbf{a}_2 \\ | & | \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2. \quad (7.2)$$

So a matrix-vector product is a **weighted sum of the columns** (a linear combination), and to know what a matrix does to all of space it is enough to know what it does to the basis. This is how you can read a transformation straight off its columns, and how you build one to do what you want.

7.3 Composition is matrix multiplication

Applying $\mathbf{B}$ then $\mathbf{A}$ to a vector is $\mathbf{A}(\mathbf{B}\mathbf{x}) = (\mathbf{A}\mathbf{B})\mathbf{x}$: **matrix multiplication is composition of maps**, performed right-to-left. It does not commute ($\mathbf{A}\mathbf{B} \neq \mathbf{B}\mathbf{A}$ in general) for the same reason physical motions do not: rotating then reflecting differs from reflecting then rotating. A neural-network layer $\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ is one such linear map (the $\mathbf{W}\mathbf{x}$ part) followed by a shift and a nonlinearity.

The mental switch

For the rest of this module, read every matrix as a verb. “What does this matrix *do* to the plane?” is answered by looking at where it sends $\mathbf{e}_1$ and $\mathbf{e}_2$ — its two columns. The next section catalogues the verbs you will meet.

8 Change of Basis for Linear Transformations

A linear map’s matrix also depends on the basis it is expressed in, even though the underlying transformation — what it does to space — does not change. Let $\mathbf{A}_B$ be T ’s matrix with respect to basis B and $\mathbf{A}_E$ its matrix with respect to basis E . Take any vector with E -coordinates $\mathbf{x}_E$, and compute where T sends it two ways. Mapping first, in E , and then converting the result to B -coordinates gives $\mathbf{\Gamma}_{BE}\mathbf{A}_E\mathbf{x}_E$. Converting first, to B -coordinates, and then mapping in B gives $\mathbf{A}_B\mathbf{\Gamma}_{BE}\mathbf{x}_E$. Both compute the very same image vector’s B -coordinates, so for every $\mathbf{x}_E$ these must agree, $\mathbf{\Gamma}_{BE}\mathbf{A}_E\mathbf{x}_E = \mathbf{A}_B\mathbf{\Gamma}_{BE}\mathbf{x}_E$, which forces

$$\mathbf{A}_E = \mathbf{\Gamma}_{EB}\mathbf{A}_B\mathbf{\Gamma}_{BE}, \quad (8.1)$$

a **similarity transform**: same map, different coordinates on both sides. The art of much of linear algebra — and of PCA — is to *choose a basis in which the map (or the data) is simplest*. For an orthonormal $\mathbf{\Gamma}_{BE}$, $\mathbf{\Gamma}_{EB} = \mathbf{\Gamma}_{BE}^T$ (Section 10), so the change of basis is itself a rigid motion.

Key idea

PCA is exactly a change of basis: it rotates the feature axes to a new orthonormal basis (the principal components) in which the data’s covariance is *diagonal* — the features become uncorrelated and ordered by variance. The machinery that finds that basis is the eigendecomposition, next.

8.1 Same Matrix, Different Meaning: Basis Change vs. Linear Map

Compare the *recipe* for Γ_{EB} (Section 4: collect the vectors of B , expressed in E , as columns) with the recipe for a linear map's matrix (Section 7.2: collect the *images of the basis vectors*, expressed in the same basis, as columns). They are the same recipe. This is not a coincidence to be explained away — it is worth building one example where it happens on purpose, to see exactly what stays the same and what does not.

Worked example

Reuse Section 4's E (canonical) and $B = \{\mathbf{b}_1, \mathbf{b}_2\}$ with $\mathbf{b}_1 = (1, 1)^\top$, $\mathbf{b}_2 = (1, -1)^\top$, so $\Gamma_{EB} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$. Now *define* a linear map $T: \mathbb{R}^2 \rightarrow \mathbb{R}^2$ by declaring where it sends the basis vectors of E , as combinations of that *same* basis: $T(\mathbf{e}_1) = \mathbf{e}_1 + \mathbf{e}_2$, $T(\mathbf{e}_2) = \mathbf{e}_1 - \mathbf{e}_2$. By the column recipe, T 's matrix in basis E is

$$\mathbf{A}_E = \begin{bmatrix} [T(\mathbf{e}_1)]_E & [T(\mathbf{e}_2)]_E \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \Gamma_{EB},$$

identical, entry for entry, because $T(\mathbf{e}_1) = \mathbf{e}_1 + \mathbf{e}_2$ happens to be the very same vector as $\mathbf{b}_1$, and $T(\mathbf{e}_2) = \mathbf{e}_1 - \mathbf{e}_2$ the very same vector as $\mathbf{b}_2$: the column that records “where T sends $\mathbf{e}_j$ ” and the column that records “ $\mathbf{b}_j$ in E -coordinates” hold the same numbers because they were built to describe the same vectors. Now apply the matrix to the same pair of numbers, $(2, 1)^\top$, under two different interpretations:

- **As a basis change.** Read $(2, 1)^\top$ as $\mathbf{v}_B$, the B -coordinates of some fixed vector $\mathbf{v}$. Then $\mathbf{v}_E = \Gamma_{EB} \mathbf{v}_B = (3, 1)^\top$. The vector $\mathbf{v}$ never moved — $(2, 1)$ and $(3, 1)$ are two rulers held up to the *same* point.
- **As a linear map.** Read $(2, 1)^\top$ as $\mathbf{w}_E$, the E -coordinates of the point $\mathbf{w}$ (i.e. $\mathbf{w}$ literally sits at $(2, 1)$ on the standard grid). Then $T(\mathbf{w})_E = \mathbf{A}_E \mathbf{w}_E = (3, 1)^\top$: T carries $\mathbf{w}$ to the *different* point $(3, 1)$, both coordinates read off the *same* ruler.

Same matrix, same input numbers, same output numbers — and two different true statements. Figure 5 draws both.

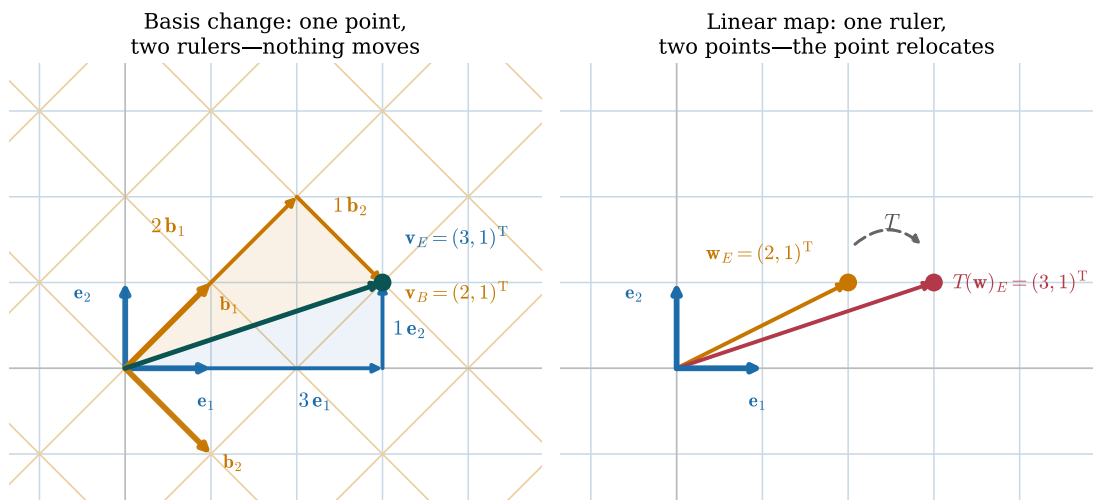


Figure 5: Left: a basis change moves *nothing*. One point is reached two ways — walk $3\mathbf{e}_1$ then $1\mathbf{e}_2$ along the square E -lattice (blue triangle), or $2\mathbf{b}_1$ then $1\mathbf{b}_2$ along the skewed B -lattice (amber triangle). Different routes, different coordinates, *same* vertex: $\mathbf{v}_E = (3, 1)^\top$ and $\mathbf{v}_B = (2, 1)^\top$

name one and the same arrow. Right: a linear map moves the point — two points, read against one grid. Both panels use the identical matrix $\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ on the identical input $(2, 1)^\top$.

Key idea

Matrix-vector multiplication, $\mathbf{y} = \mathbf{A}\mathbf{x}$, is pure arithmetic: it does not know whether $\mathbf{x}$ and $\mathbf{y}$ are two descriptions of one vector or the coordinates of two different vectors. That meaning comes entirely from what you declare the symbols to stand for. One structural difference does survive, though: a basis-change matrix is *always* invertible (its columns are a basis, by construction), because you can always translate back. A linear map's matrix need not be — a projection (Section 9) collapses a whole direction to zero, and there is no ruler that undoes that.

8.2 Choosing an Easy Basis: the Conjugation Trick

Equation (8.1) is easy to read as bad news — a map's matrix is not even a fixed object, it depends on the ruler you hold up to it. Read the other way round it is the single most useful technique in this module, because *you get to choose the ruler*.

Suppose you must write down the matrix of some geometric operation that is awkward in the canonical basis: a mirror reflection in a line through the origin at some angle, a rotation about an axis that is not x , y or z , a projection onto an arbitrary plane. Working out where such a map sends $\mathbf{e}_1$, $\mathbf{e}_2$ and $\mathbf{e}_3$ directly is unpleasant trigonometry. So do not. Instead:

Key idea

The conjugation trick. Choose a basis B adapted to the geometry of the map — put its vectors along the directions the map treats specially (the mirror line, the rotation axis, the subspace you project onto, and their perpendiculars). In that basis the matrix $\mathbf{A}_B$ is diagonal, or block-diagonal, and you can write it down by inspection. Then transport it back:

$$\mathbf{A}_E = \mathbf{\Gamma}_{EB} \mathbf{A}_B \mathbf{\Gamma}_{BE}, \quad \text{with} \quad \mathbf{\Gamma}_{EB} = [\mathbf{b}_1 \ \mathbf{b}_2 \ \cdots \ \mathbf{b}_n], \quad \mathbf{\Gamma}_{BE} = \mathbf{\Gamma}_{EB}^{-1}. \quad (8.2)$$

You never compute $\mathbf{A}_E$ directly. You *assemble* it.

Two practical points before the examples.

Read it right to left. Matrix products act on the right-hand vector first, so (8.2) is a three-step pipeline executed from the right: translate the input into B -coordinates, act there, translate the answer back. The diagram below lays out that order explicitly, because the written formula reverses it.

If B is orthonormal, there is no inverse to compute. Choosing the adapted basis to be orthonormal makes $\mathbf{\Gamma}_{EB} = \mathbf{Q}$ orthogonal, so $\mathbf{\Gamma}_{BE} = \mathbf{Q}^{-1} = \mathbf{Q}^\top$ (Section 10) and (8.2) becomes

$$\mathbf{A}_E = \mathbf{Q} \mathbf{A}_B \mathbf{Q}^\top. \quad (8.3)$$

Transposing is free; inverting is not. Every example below uses an orthonormal B , and so does every serious application — which is exactly why Gram–Schmidt (Section 6.6) was worth the effort.

1 — Reflection in a line at angle φ

Mirror the plane in the line through the origin making angle φ with the x -axis. In the canonical basis this is not obvious. In the adapted basis it is trivial: take $\mathbf{b}_1 = (\cos \varphi, \sin \varphi)^\top$ along the mirror and $\mathbf{b}_2 = (-\sin \varphi, \cos \varphi)^\top$ across it. A mirror leaves the first alone and

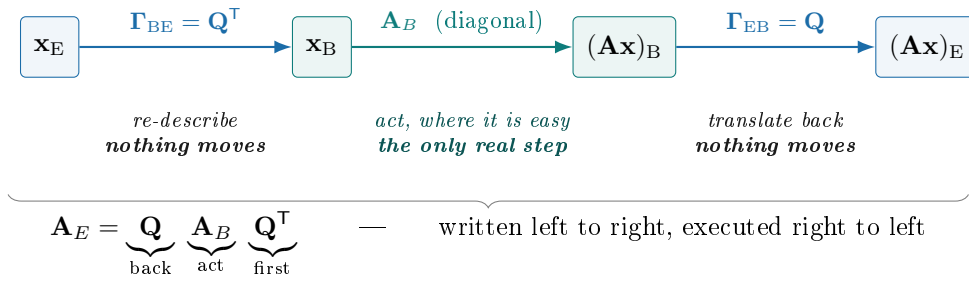


Figure 6: The conjugation trick as a pipeline. The outer two steps are pure relabelling: the arrow in space is untouched, only the coordinates describing it change. All of the geometry happens in the middle, in the basis where the map is diagonal. This is the same square as the derivation of (8.1), walked in one direction.

negates the second, so

$$\mathbf{A}_B = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad \mathbf{Q} = \Gamma_{EB} = \begin{bmatrix} \cos \varphi & -\sin \varphi \\ \sin \varphi & \cos \varphi \end{bmatrix}.$$

Conjugating, $\mathbf{A}_E = \mathbf{Q}\mathbf{A}_B\mathbf{Q}^T$, and the product collapses by the double-angle identities to

$$\mathbf{A}_E = \begin{bmatrix} \cos 2\varphi & \sin 2\varphi \\ \sin 2\varphi & -\cos 2\varphi \end{bmatrix}.$$

For $\varphi = 30^\circ$ that is $\begin{bmatrix} 1/2 & \sqrt{3}/2 \\ \sqrt{3}/2 & -1/2 \end{bmatrix}$. Check it: $\det = -1$ (a reflection, Section 10), the mirror direction is fixed, and the perpendicular is negated. Note the 2φ — reflecting in a line tilted by φ rotates the *axes of the matrix* by 2φ , a fact that is obvious from the picture and mysterious from the entries.

2 — Rotation by θ about an arbitrary axis in $\mathbb{R}^3$

This is the case that matters for the Robotics & Bionics unit, and the one where doing it directly is genuinely hard.

Let $\mathbf{a}$ be a unit vector along the desired axis. Build any orthonormal pair $\mathbf{u}, \mathbf{v}$ perpendicular to it (take any vector not parallel to $\mathbf{a}$, subtract its component along $\mathbf{a}$, normalise, then set $\mathbf{v} = \mathbf{a} \times \mathbf{u}$). In the basis $B = \{\mathbf{a}, \mathbf{u}, \mathbf{v}\}$ the rotation fixes the first axis and turns the other two into each other, so it is the familiar 2×2 rotation sitting in a corner:

$$\mathbf{A}_B = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}, \quad \mathbf{Q} = [\mathbf{a} \ \mathbf{u} \ \mathbf{v}].$$

Then $\mathbf{A}_E = \mathbf{Q}\mathbf{A}_B\mathbf{Q}^T$ is the rotation about $\mathbf{a}$ expressed in canonical coordinates. Expanding the product symbolically reproduces **Rodrigues' rotation formula**,

$$\mathbf{R} = \cos \theta \mathbf{I} + \sin \theta [\mathbf{a}]_{\times} + (1 - \cos \theta) \mathbf{a}\mathbf{a}^T,$$

which is usually presented as something to be memorised. It is not: it is (8.3) with the multiplication carried out.

A check worth doing by hand. Take $\mathbf{a} = \frac{1}{\sqrt{3}}(1, 1, 1)^\top$ and $\theta = 120^\circ$. The recipe returns

$$\mathbf{A}_E = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix},$$

exactly — the matrix that cycles $\mathbf{e}_1 \rightarrow \mathbf{e}_2 \rightarrow \mathbf{e}_3 \rightarrow \mathbf{e}_1$. That is correct and obvious in hindsight: the diagonal of the cube is precisely the axis about which a third of a turn permutes the three coordinate axes. An assembly of irrational entries has produced an integer matrix, which is a good sign you have not made an arithmetic slip.

3 — Projection onto a plane, and a familiar face

Project $\mathbb{R}^3$ orthogonally onto the plane $x + y + z = 0$, with unit normal $\mathbf{n} = \frac{1}{\sqrt{3}}(1, 1, 1)^\top$. Choose $\mathbf{q}_1, \mathbf{q}_2$ an orthonormal basis of the plane and take $B = \{\mathbf{q}_1, \mathbf{q}_2, \mathbf{n}\}$. A projection keeps what lies in the plane and destroys what does not:

$$\mathbf{A}_B = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{A}_E = \mathbf{Q}\mathbf{A}_B\mathbf{Q}^\top = \mathbf{I} - \mathbf{n}\mathbf{n}^\top = \frac{1}{3} \begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}.$$

The zero in $\mathbf{A}_B$ is where the non-invertibility of a projection lives, plainly visible; $\text{tr}(\mathbf{A}_E) = 2$ counts the surviving dimensions.

Now notice what $\mathbf{Q}\mathbf{A}_B\mathbf{Q}^\top$ actually computes. Writing $\mathbf{Q}_k = [\mathbf{q}_1 \ \mathbf{q}_2]$ for just the columns that survive, it equals $\mathbf{Q}_k\mathbf{Q}_k^\top$ — and that is the least-squares hat matrix of Annex A in the case of an orthonormal basis, since $\mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$ reduces to $\mathbf{X}\mathbf{X}^\top$ when $\mathbf{X}^\top\mathbf{X} = \mathbf{I}$. Regression's projector is not a separate construction; it is this trick applied to $\text{col}(\mathbf{X})$, with $(\mathbf{X}^\top\mathbf{X})^{-1}$ doing the extra work of orthonormalising a basis that was handed to you crooked.

Key idea

Why this keeps coming back. Almost everything later in this module is the same move with a different way of finding B :

- **Eigendecomposition** (Section 12): $\mathbf{A} = \mathbf{P}\mathbf{\Lambda}\mathbf{P}^{-1}$ — the easy basis is the eigenbasis, found rather than chosen, and there the map is pure scaling.
- **SVD** (Section 13): $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ — two different bases, one for the input space and one for the output, because a non-square map has no single basis to conjugate with.
- **PCA** (Section 14.2): the easy basis is the one in which the covariance is diagonal, i.e. in which the features are uncorrelated.
- **Least squares** (Section 14.1): the easy basis is an orthonormal basis of $\text{col}(\mathbf{X})$, and $\mathbf{A}_B$ keeps the first n coordinates and discards the rest.

Four constructions that look unrelated in a formula sheet are one idea: *find the basis in which the thing is diagonal, work there, come back.*

Exercises

1. Build the reflection in the line $y = x$ by conjugation ($\varphi = 45^\circ$) and confirm you get $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, which swaps the coordinates — as a mirror in $y = x$ must.
2. Using the same adapted basis as Example 1 but with $\mathbf{A}_B = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$, build the map that

stretches the plane by a factor 3 *along* the line at angle φ and leaves the perpendicular direction alone. Verify your matrix is symmetric, and explain why it had to be.

3. Verify the 120° claim in Example 2 numerically in NumPy: construct $\mathbf{u}$ and $\mathbf{v}$, assemble $\mathbf{QA}_B\mathbf{Q}^\top$, and check it equals the permutation matrix to machine precision. Then confirm $\det = +1$ and $\mathbf{A}_E\mathbf{a} = \mathbf{a}$.
4. Rotate by 90° about the axis $\mathbf{a} = \frac{1}{\sqrt{2}}(1, 0, 1)^\top$ by conjugation. Check your answer is orthogonal with $\det = +1$, and that applying it four times returns the identity.
5. Show that if $\mathbf{A}_B$ is symmetric and $\mathbf{Q}$ is orthogonal then $\mathbf{QA}_B\mathbf{Q}^\top$ is symmetric. Which of the three examples above are therefore guaranteed symmetric, and which are not?
6. In Example 3, compute $\mathbf{A}_E$ for the projection onto the *line* spanned by $\mathbf{n}$ instead of the plane (use $\mathbf{A}_B = \text{diag}(0, 0, 1)$). Confirm the two projectors sum to $\mathbf{I}$, and say what that means geometrically.

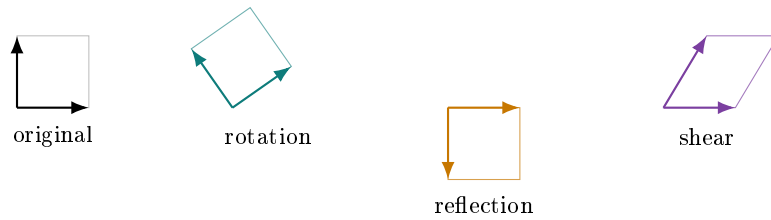
9 The Transformation Zoo

Table 1 collects the linear maps you will meet, in 2-D, each read through its columns (where it sends $\mathbf{e}_1$ and $\mathbf{e}_2$).

Table 1: The transformation zoo. Every entry is read the same way: the first column is where $\mathbf{e}_1$ lands, the second where $\mathbf{e}_2$ lands.

Map	Matrix	What it does
Scaling	$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$	stretches axis x by s_x , axis y by s_y
Rotation by θ	$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$	turns every vector by θ about the origin
Reflection (in x -axis)	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	flips across the x -axis (a mirror)
Shear (horizontal)	$\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$	slides points sideways in proportion to height
Projection (onto x -axis)	$\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$	flattens the plane onto a line (information lost)

The rotation matrix is worth memorising; it is the 2-D case of the rotation matrices $\mathbf{R}_x, \mathbf{R}_y, \mathbf{R}_z$ in the Robotics & Bionics unit. Note the projection's second column is zero: it sends $\mathbf{e}_2$ to the origin, so a whole direction collapses — projections are *not* invertible.



Exercises

1. Write the matrix that rotates by 90° . Apply it to $(1,0)^\top$ and $(0,1)^\top$ and confirm the result matches its columns.
2. What single matrix scales x by 2, leaves y unchanged, *and* then rotates by 90° ? (Remember: compose right-to-left.)
3. Show that the projection $\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$ applied twice equals itself (it is *idempotent*), and explain why geometrically.

10 Orthogonal Matrices and Rigid Motions

A matrix $\mathbf{Q}$ is **orthogonal** if its columns are mutually perpendicular unit vectors, equivalently

$$\mathbf{Q}^\top \mathbf{Q} = \mathbf{I} \iff \mathbf{Q}^{-1} = \mathbf{Q}^\top. \quad (10.1)$$

Such maps **preserve lengths and angles**: $\|\mathbf{Q}\mathbf{x}\| = \|\mathbf{x}\|$ and $(\mathbf{Q}\mathbf{x})^\top (\mathbf{Q}\mathbf{y}) = \mathbf{x}^\top \mathbf{y}$. They are the *rigid motions* — they move a shape without distorting it. Their determinant is

$$\det \mathbf{Q} = \pm 1 : \quad +1 \text{ a rotation}, \quad -1 \text{ a reflection}. \quad (10.2)$$

Key idea

This is exactly the $SO(3)$ structure from the Robotics & Bionics unit: a robot's orientation is an orthogonal matrix with $\det = +1$ (a pure rotation), and its inverse is just its transpose — which is why undoing a rotation is free. The same algebra that orients a surgical tool orients a coordinate frame in PCA.

11 Inversion, Determinant, Rank, and Solving $\mathbf{Ax} = \mathbf{b}$

The **determinant** of a square matrix measures how it scales area (2-D) or volume (3-D), and its *sign* records orientation. $\det \mathbf{A} = 0$ means the map squashes space into a lower dimension (a line or a point), so it destroys information and **cannot be inverted** — the matrix is *singular*. When $\det \mathbf{A} \neq 0$ the **inverse** $\mathbf{A}^{-1}$ undoes the map, $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$.

The **rank** is the dimension of the *image* — the column space, the set of outputs the map can actually reach — equivalently the number of independent directions that survive it. A full-rank $n \times n$ matrix is invertible; a rank-deficient one has collapsed at least one direction (like the projection above). In data terms, rank deficiency means columns (features) are linearly dependent — redundant information — which is precisely what dimensionality reduction exploits.

11.1 Solving a linear system

Whether $\mathbf{Ax} = \mathbf{b}$ has a solution — and how many — depends on $\mathbf{A}$'s shape and rank:

- **Square, full rank** ($\det \mathbf{A} \neq 0$). A unique solution always exists, $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ — inverting the map.

- **More equations than unknowns (overdetermined).** Generically no exact solution exists; Section 14 finds the closest one by projection instead.
- **Rank-deficient or underdetermined.** Either no solution exists (the equations are inconsistent) or infinitely many do (any point in an entire solution subspace works) — rank tells you which, and dimensionality reduction exploits exactly this redundancy. When there are infinitely many, the pseudoinverse of Section 14.1 settles the ambiguity by returning the solution of smallest norm.

In practice $\mathbf{A}^{-1}$ is rarely computed explicitly even when it exists; solvers use Gaussian elimination or a factorisation (Gram–Schmidt/QR, Section 6.6, or the SVD, Section 13) instead, for speed and numerical stability.

Exercises

1. Compute $\det\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$ and state the area-scaling factor of the map.
2. Verify that the rotation matrix has determinant 1 and the reflection matrix determinant -1 .
3. Why does the projection onto the x -axis have rank 1 and determinant 0? What does that say about inverting it?
4. Does $\mathbf{A}\mathbf{x} = \mathbf{b}$ have a solution for $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$, $\mathbf{b} = (1, 2)^\top$? What about $\mathbf{b} = (1, 3)^\top$? Explain using the rank of $\mathbf{A}$.

12 Eigenvalues and Eigenvectors

Most directions are rotated by a map. An **eigenvector** is a special direction the map only *stretches*, not turns:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}, \quad (12.1)$$

where the scalar λ is its **eigenvalue** (the stretch factor). Eigenvectors are the map's invariant axes; eigenvalues say how much it scales along them. If an $n \times n$ matrix has n independent eigenvectors, stacking them as the columns of $\mathbf{P}$ gives the **diagonalisation**

$$\mathbf{A} = \mathbf{P}\mathbf{\Lambda}\mathbf{P}^{-1}, \quad \mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n), \quad (12.2)$$

which is just (8.1) read backwards — $\mathbf{P}$ playing the role of $\mathbf{\Gamma}_{\text{EB}}$ for B the eigenbasis, where T 's matrix is the diagonal $\mathbf{\Lambda}$ — *in the eigenvector basis the map is pure scaling*. This makes powers trivial ($\mathbf{A}^k = \mathbf{P}\mathbf{\Lambda}^k\mathbf{P}^{-1}$) and is the engine behind solving linear dynamical systems — including the state-space models of the Control unit.

The spectral theorem (symmetric matrices)

If $\mathbf{A}$ is **symmetric** ($\mathbf{A} = \mathbf{A}^\top$) then its eigenvalues are real and its eigenvectors can be chosen **orthonormal**: $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ with $\mathbf{Q}$ orthogonal. Covariance matrices, Gram matrices and graph Laplacians are all symmetric — so this case is the one ML uses most, and it is what makes PCA's new axes orthogonal.

13 The Singular Value Decomposition

Eigendecomposition needs a square matrix. The **singular value decomposition (SVD)** generalises it to *any* $m \times n$ matrix and is, arguably, the most important factorisation in data science:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top, \quad (13.1)$$

with $\mathbf{U}$ ($m \times m$) and $\mathbf{V}$ ($n \times n$) orthogonal and $\mathbf{\Sigma}$ diagonal with non-negative **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$. Geometrically every linear map is therefore **rotate** ($\mathbf{V}^\top$) $\rightarrow$ **scale along axes** ($\mathbf{\Sigma}$) $\rightarrow$ **rotate** ($\mathbf{U}$): a rigid turn, an axis-aligned stretch, another rigid turn. The singular values are the lengths of the axes of the ellipse that the unit circle maps to.

Key idea

Low-rank approximation. Keeping only the largest k singular values, $\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$, is the *best* rank- k approximation of $\mathbf{A}$ (Eckart–Young). This single fact powers image compression, noise removal, recommender systems and PCA. In the lab you will compress a microscopy-style image by discarding small singular values and watch the picture survive on a fraction of the data — a direct bridge to the image module.

14 Two Payoffs: Least Squares and PCA

14.1 Linear regression is a projection

Fitting $\mathbf{X}\mathbf{w} \approx \mathbf{y}$ when there are more equations than unknowns has no exact solution, so we find the $\mathbf{w}$ making $\mathbf{X}\mathbf{w}$ as close as possible to $\mathbf{y}$ — i.e. the **projection** of $\mathbf{y}$ onto the column space of $\mathbf{X}$. The residual must be orthogonal to every column, $\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) = \mathbf{0}$, which rearranges to the **normal equation**

$$\boxed{\mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y} \quad \Rightarrow \quad \mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.} \quad (14.1)$$

This is the closed-form linear regression of Géron’s Chapter 4 — and it is nothing but Section 6.5’s projection, done in many dimensions.

Substituting that $\mathbf{w}$ back makes the projection explicit as a matrix. The fitted values are

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} = \underbrace{\mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top}_{\mathbf{P}} \mathbf{y}, \quad \mathbf{r} = \mathbf{y} - \hat{\mathbf{y}} = (\mathbf{I} - \mathbf{P}) \mathbf{y}, \quad (14.2)$$

so $\mathbf{P}$ takes any target and returns its shadow in the column space of $\mathbf{X}$ — the space of every prediction the model is capable of making (Figure 7). Two properties earn it the name. It is *symmetric*, $\mathbf{P}^\top = \mathbf{P}$, and *idempotent*, $\mathbf{P}^2 = \mathbf{P}$: projecting something that already lies in the subspace must change nothing, which is what “shadow” has to mean. And $\mathbf{I} - \mathbf{P}$ projects onto the orthogonal complement, so $\mathbf{y} = \hat{\mathbf{y}} + \mathbf{r}$ is exactly the “along plus perpendicular” split of Section 6.5 — onto a subspace now, rather than onto a line.

The correspondence with that earlier formula is exact, not an analogy. Put a single column $\mathbf{x}$ into $\mathbf{X}$: then $\mathbf{X}^\top \mathbf{X}$ is the scalar $\mathbf{x}^\top \mathbf{x}$, and

$$\mathbf{P}\mathbf{y} = \mathbf{x} \frac{1}{\mathbf{x}^\top \mathbf{x}} \mathbf{x}^\top \mathbf{y} = \frac{\mathbf{x}^\top \mathbf{y}}{\mathbf{x}^\top \mathbf{x}} \mathbf{x},$$

which is (6.6) character for character. One column is a line, n columns are a subspace, and the operator is the same.

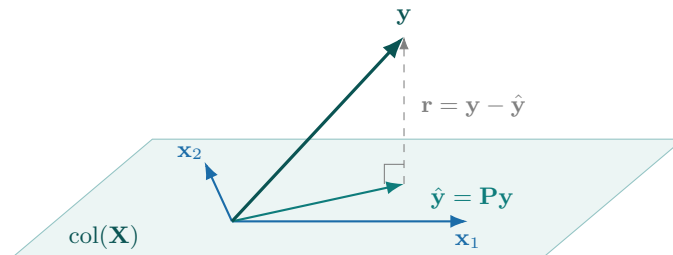


Figure 7: The geometry of least squares. The columns of $\mathbf{X}$ span a subspace, and every prediction the model can make lives in it. The target $\mathbf{y}$ generally does not — that is precisely what “more equations than unknowns” means. The best fit is its shadow $\hat{\mathbf{y}} = \mathbf{P}\mathbf{y}$, and the residual $\mathbf{r}$ stands perpendicular to the subspace, hence orthogonal to every column: the normal equation, read off a picture.

In practice nobody forms $(\mathbf{X}^\top \mathbf{X})^{-1}$. Squaring the matrix squares its conditioning, so the normal equation is solved instead through the QR factorisation of Section 6.6 or, more commonly, through the SVD of Section 13. Writing $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, the least-squares solution is $\mathbf{w} = \mathbf{X}^+ \mathbf{y}$ with the **pseudoinverse**

$$\mathbf{X}^+ = \mathbf{V}\mathbf{\Sigma}^+ \mathbf{U}^\top, \quad (14.3)$$

where $\mathbf{\Sigma}^+$ is built by transposing $\mathbf{\Sigma}$ and replacing each non-zero singular value σ_i by $1/\sigma_i$, leaving the zeros alone. That last clause is the point: $\mathbf{X}^+$ exists even when $\mathbf{X}^\top \mathbf{X}$ is singular — when features are redundant and infinitely many $\mathbf{w}$ fit equally well, it returns the one of smallest norm rather than failing. This is exactly what `np.linalg.pinv` and `scipy.linalg.lstsq` compute, and therefore what Scikit-Learn’s `LinearRegression` does when you call `fit`.

Key idea

Ridge regression is one line of linear algebra. Penalising large coefficients with the squared L2 norm of Section 6.3 — minimising $\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \alpha\|\mathbf{w}\|_2^2$ — changes the normal equation to

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \alpha \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (14.4)$$

Read it through the spectral theorem (Section 12): $\mathbf{X}^\top \mathbf{X}$ is symmetric with eigenvalues $\lambda_i \geq 0$, and adding $\alpha \mathbf{I}$ shifts every one of them to $\lambda_i + \alpha$. Three consequences at once. The matrix becomes invertible whatever the data did, since no eigenvalue can sit at zero any more. The condition number $\lambda_{\max}/\lambda_{\min}$ falls, so the solve is numerically better behaved. And the shrinkage is uneven: a direction with large λ_i is barely touched, while a direction the data barely explores is pulled hard toward zero — regularisation suppresses exactly the directions the data does not constrain. Lasso replaces the L2 penalty with L1, which has no closed form but drives coefficients exactly to zero. Both are Géron’s Chapter 4.

14.2 PCA is an eigen/SVD problem

Principal-component analysis finds the orthonormal directions of greatest variance in the data and keeps only the first few. Every piece of machinery in this module shows up in that one sentence.

Centre first — this is not optional. PCA measures variance *about the mean*, so subtract the column means before anything else. On uncentred data the first component simply points at the mean, which is a fact about where the cloud sits rather than how it is shaped. Scikit-Learn’s `PCA` centres for you; a hand-rolled version that forgets to will produce confident nonsense.

For centred $\mathbf{X}$ (m samples $\times$ n features) the **covariance matrix** is

$$\mathbf{C} = \frac{1}{m} \mathbf{X}^\top \mathbf{X}, \quad (14.5)$$

which is symmetric, so by the spectral theorem (Section 12) its eigenvalues are real and its eigenvectors can be taken orthonormal. They are also non-negative, because $\mathbf{v}^\top \mathbf{C} \mathbf{v} = \frac{1}{m} \|\mathbf{X} \mathbf{v}\|^2 \geq 0$ — a covariance matrix is **positive semi-definite**, and an eigenvalue *is* a variance, so it could hardly be otherwise. Those eigenvectors are the **principal components**; the i -th eigenvalue λ_i is the variance of the data along the i -th one.

The SVD gets there without ever forming $\mathbf{C}$. If $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ then $\mathbf{X}^\top\mathbf{X} = \mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^\top$, so the principal components are the right singular vectors $\mathbf{V}$ and

$$\lambda_i = \frac{\sigma_i^2}{m}. \quad (14.6)$$

This is how it is computed in practice — squaring $\mathbf{X}$ to build $\mathbf{C}$ would square the conditioning, the same argument as for least squares above.

How many components to keep. The eigenvalues are variances in the same units, and they sum to the total variance in the data, so each one's share is the **explained variance ratio**

$$\frac{\lambda_i}{\sum_j \lambda_j} = \frac{\sigma_i^2}{\sum_j \sigma_j^2}, \quad (14.7)$$

which is what `pca.explained_variance_ratio_` reports. The usual recipe is to take the smallest k whose cumulative ratio clears a threshold — Géron uses 95% — rather than choosing k by eye.

Key idea

Projection, and the way back. Stacking the top k components as the columns of $\mathbf{V}_k$, the reduced data is $\mathbf{X}\mathbf{V}_k$ ($m \times k$) — the coordinates of each sample in the new basis, which is exactly the change of basis of Section 4 applied one sample at a time. Mapping back gives $\hat{\mathbf{X}} = \mathbf{X}\mathbf{V}_k\mathbf{V}_k^\top$, and $\mathbf{V}_k\mathbf{V}_k^\top$ is precisely the projection matrix of Section 6.5, now onto a k -dimensional subspace instead of a line. So PCA compresses by projecting, the discarded variance $\sum_{i>k} \lambda_i$ is the reconstruction error, and Eckart–Young (Section 13) says no other k -dimensional subspace does better. One caveat before you use it: PCA chases variance, and variance depends on units. Features on wildly different scales (millivolts beside kilograms) must be standardised first, or the large-numbered feature will be declared the first principal component for no better reason than its units.

PCA is thus a change of basis (Section 8) chosen by the SVD (Section 13) so that the covariance becomes diagonal — the features come out uncorrelated and ordered by variance. It is Géron's Chapter 7.

Exercises

1. Find the eigenvalues and eigenvectors of $\begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$ and of $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$. Which directions are invariant?
2. For the symmetric matrix above, verify the eigenvectors are orthogonal (spectral theorem).
3. Explain in one sentence why the normal equation is a projection statement.
4. Show that $\mathbf{P} = \mathbf{X}(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$ of (14.2) is symmetric and idempotent, and check that $\mathbf{I} - \mathbf{P}$ is too.
5. What is $\mathbf{P}$ when $\mathbf{X}$ is square and invertible? Compute it, and explain in one sentence why that answer is the only one the geometry allows.
6. State what the singular values of a data matrix tell you about its principal components.
7. A data matrix has singular values $\sigma = (6, 3, 2, 1)$. Give the explained variance ratio of each component and the smallest number of components retaining at least 95% of the variance.
8. Two features of a dataset are duplicates of each other. What does that do to the

eigenvalues of $\mathbf{X}^\top \mathbf{X}$, why does $(\mathbf{X}^\top \mathbf{X})^{-1}$ then fail, and what do (14.3) and (14.4) each do about it?

9. Show that $\mathbf{V}_k \mathbf{V}_k^\top$ is idempotent ($(\mathbf{V}_k \mathbf{V}_k^\top)^2 = \mathbf{V}_k \mathbf{V}_k^\top$) and explain why any projection matrix must be.

15 PageRank: An Eigenvector-Centrality Case Study

Eigenvectors do not just diagonalise a matrix — they can *rank* things. Google’s original PageRank algorithm poses “how important is this web page” as an eigenvalue problem, and the identical algebra ranks genes in a regulatory network or hub regions in a brain-connectivity graph.

15.1 Importance as a fixed point

Model a set of pages as a directed graph, page j pointing at page i whenever it links to it. Define importance recursively: *a page is important if important pages link to it*, each linker splitting its importance evenly across all of its outgoing links. Writing $L_{ij} = 1/(\text{number of pages } j \text{ links to})$ when j links to i and 0 otherwise, the vector of importances $\mathbf{r}$ (nonnegative, summing to 1 — a probability distribution over pages) must satisfy

$$\mathbf{L}\mathbf{r} = \mathbf{r}, \quad (15.1)$$

precisely the eigenvector equation of Section 12 with eigenvalue $\lambda = 1$. Every column of $\mathbf{L}$ sums to 1 (each page redistributes all of its importance), so $\mathbf{L}$ is **column-stochastic** — exactly the transition matrix of a random walk in which a “random surfer” follows a uniformly random outgoing link at each step, and $\mathbf{r}$ is that walk’s **stationary distribution**: the long-run fraction of time the surfer spends on each page.

15.2 Finding $\mathbf{r}$: power iteration

Solving (15.1) directly does not scale to millions of pages. Instead, start from any distribution $\mathbf{r}_0$ and repeatedly apply $\mathbf{L}$,

$$\mathbf{r}_{k+1} = \mathbf{L}\mathbf{r}_k. \quad (15.2)$$

Because $\mathbf{L}$ is column-stochastic its dominant eigenvalue is exactly 1, so this **power iteration** converges (generically) to the $\mathbf{r}$ we want — without ever computing eigenvalues explicitly. It is the same trick used to approximate a dominant eigenvector of any matrix.

15.3 Dead ends, traps and damping

Two structural problems break the simple story. A page with no outgoing links (a **dead end**) has a column of zeros, not a column summing to 1, so it drains probability out of the system every step. A small closed loop of pages (a **trap**) can instead absorb all of the probability and starve everything else. Both are repaired the same way: assume that at every step the surfer follows a link with probability d and otherwise jumps to a uniformly random page,

$$\mathbf{M} = d\mathbf{L} + \frac{1-d}{n}\mathbf{J}, \quad \mathbf{J} = \text{the all-ones matrix}, \quad (15.3)$$

with Google’s own historical choice $d \approx 0.85$. By construction $\mathbf{M}$ is column-stochastic whatever $\mathbf{L}$ looks like, so it always has a well-defined stationary distribution — however the underlying link graph is shaped.

Key idea

This is one instance of **eigenvector centrality**: rank the nodes of *any* weighted graph by “important nodes are those pointed to by important nodes,” i.e. by the dominant eigenvector of its (damped) connectivity matrix. The identical algebra ranks web pages, genes in a regulatory network, or — in the lab — hub regions of a synthetic brain connectivity network, identifying the regions most central to information flow.

Exercises

1. For a 3-page network where every page links to both others (and not to itself), write $\mathbf{L}$ and confirm the uniform distribution $\mathbf{r} = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})^\top$ solves (15.1).
2. For a 2-page network where page A links only to B and B links only to A, write $\mathbf{L}$ and confirm $\mathbf{r} = (\frac{1}{2}, \frac{1}{2})^\top$ solves (15.1).
3. Explain why a dead-end page breaks the column-stochastic property of $\mathbf{L}$, and how the damping term in (15.3) repairs it.
4. With $d = 0.85$ and $n = 100$ pages, what fraction of a page’s importance in $\mathbf{M}$ comes from the random-jump term rather than from its incoming links?

16 The Lab, and Where This Leads in Géron

Key idea

Companion Colab notebooks. After the theory, the session moves to Colab. Module 2 ships *three* self-contained notebooks — together a complete, biomedical reference, with no need to consult any other material:

- `m2_vectors_transforms` — vector spaces, basis and coordinates; the scalar product, norms and projection; Gram–Schmidt orthonormalization (Section 6.6), including a vectorised speed-up; matrices as linear maps; the transformation zoo with live 2-D visualisations; orthonormal matrices and rigid motions; determinant, rank and change of basis; and a bonus section reproducing Holbein’s anamorphic-skull illusion as a worked change of basis;
- `m2_eigen_svd_pca` — eigenvectors and diagonalisation; the SVD geometry; image compression by low-rank SVD; least squares via the normal equation; and PCA on the breast-cancer data, compared to Scikit-Learn;
- `m2_pagerank_networks` — PageRank as an eigenvalue problem (Section 15); power iteration; dead ends, traps and the damping parameter; and a bioengineering capstone ranking hub regions of a synthetic brain-connectivity network.

The first two are each a superset of Géron’s `math_linear_algebra` notebook, re-ordered around geometry and worked on biomedical data; all three end their sections with exercises and worked solutions.

Where this unlocks the book. Projection gives the normal equation of **Chapter 4** (training linear models); Gram–Schmidt/QR gives the numerically stable way to actually solve it; the eigen/SVD machinery gives **Chapter 7** (dimensionality reduction / PCA); matrices-as-maps is exactly a neural-network layer in **Chapters 9–10**; and PageRank is the same eigenvector idea turned into a ranking algorithm — the ancestor of every “importance score” built on a graph. *Cross-links*: the rotation and orthonormal-matrix material is the $SO(3)$ of the Robotics & Bionics unit, and the eigen-machinery reappears in the state-space analysis of the Control

unit.

The one-paragraph summary

A vector space is any set with an addition and a scaling obeying the usual rules — $\mathbb{R}^n$ is the example we compute with, but polynomials and matrices qualify too, and nothing in the theory notices the difference. A basis is the minimal set of vectors needed to reach every point by linear combination, and $\mathbf{T}_{BE}$ translates one basis's coordinates into another's by collecting the second basis, expressed in the first, as columns — coordinates, including the row/column arrays you already know, are just weights relative to some basis. The dot product measures length, angle and overlap, and projection splits a vector into “along” and “perpendicular” parts — which is least squares; Gram–Schmidt repeats that split to build an orthonormal basis, and hence a QR factorisation, out of any one. A linear transformation is a map respecting addition and scaling, and every one of them — in finite dimensions — is a matrix built by the very same column recipe as a basis-change matrix: send the second basis's vectors where you like instead of merely relabelling them, and the identical arithmetic now moves points instead of just renaming them. Orthogonal matrices are rigid motions ($\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$, $\det = \pm 1$); the determinant is the area/volume scale and zero means non-invertible, which is exactly when $\mathbf{Ax} = \mathbf{b}$ stops having a unique solution. Eigenvectors are the directions a map only stretches; for symmetric matrices they are orthonormal, and the same fixed-point idea — iterated on a column-stochastic matrix instead of diagonalised — is PageRank. The SVD writes any map as rotate–scale–rotate and gives the best low-rank approximation — the basis of compression and of PCA. Choose the right basis and the problem becomes easy.